ODISHA ADARSHA VIDYALAYA SANGATHAN (OAVS)

TGT COMPUTER TEACHER

MCQ Question Bank — Language Features

176 Practice Questions

Scope Rules • Static Data / Methods / Blocks • Modifiers • Abstract Classes
Interfaces • Inner Classes • Wrapper Classes • Packages & Access • Imports • User-Defined Packages

Every question includes the correct answer and a concise explanation.
Answer options are randomized and balanced across A / B / C / D.

Table of Contents

1	Scope Rules	12 Qs
2	Static Data	10 Qs
3	Static Methods	10 Qs
4	Static Blocks	10 Qs
5	Class, Method, Data Member and Variable Modifiers	15 Qs
6	Abstract Classes	12 Qs
7	Interfaces	12 Qs
8	Inner Classes	10 Qs
9	Wrapper Classes	10 Qs
10	Packages	8 Qs
11	Package Access	8 Qs
12	Importing Packages and Classes	8 Qs
13	User-Defined Packages	8 Qs
14	Comparison Questions	11 Qs
15	Code, Output and Compilation Questions	12 Qs
16	Difficult & Tricky Questions	10 Qs
17	Mixed OAVS TGT Practice Set	10 Qs
	Total	**176 Qs**

Section 1 — Scope Rules

12 Questions

Q1 | Scope Rules | Concept of Scope

What does 'scope' of a variable refer to in Java?

A. The data type the variable can hold
B. The memory address assigned to the variable
C. The number of times a variable can be used
D. The region of the program within which the variable can be accessed

Ans: D

Explanation: Scope defines the block or region of code in which a declared variable is visible and accessible.

Q2 | Scope Rules | Local Scope

A variable declared inside a method body is called a:

A. Local variable
B. Instance variable
C. Class variable
D. Global variable

Ans: A

Explanation: Variables declared inside a method, constructor, or block are local variables, scoped to that block only.

Q3 | Scope Rules | Local Scope

Local variables in Java are stored in:

A. Stack memory, for the duration of the method call
B. Heap memory, for the life of the object
C. Static memory, for the life of the program
D. Constant pool, permanently

Ans: A

Explanation: Local variables exist only while their enclosing method/block executes and are held on the stack.

Q4 | Scope Rules | Member Scope

A variable declared inside a class but outside any method is known as:

A. A local variable
B. A parameter variable
C. An instance (member) variable
D. A block variable

Ans: C

Explanation: Variables declared directly in the class body are member/instance variables, scoped to the whole class.

Q5 | Scope Rules | Variable Visibility

What happens if a local variable is used before it is initialized?

A. A compile-time error occurs
B. It is automatically initialized to zero
C. It is initialized to null
D. The program runs but prints garbage value

Ans: A

Explanation: Unlike instance variables, local variables are not given default values, so using them uninitialized causes a compiler error.

Q6 | Scope Rules | Block Scope

Consider:
if (true) {
int x = 5;
}
System.out.println(x);
What is the result?

A. Prints 5
B. Prints 0
C. Runtime exception
D. Compilation error, x is out of scope

Ans: D

Explanation: x is declared inside the if-block, so its scope ends with that block; it cannot be accessed outside it.

Q7 | Scope Rules | Nested Scope

Can a local variable declared in an outer block be accessed inside an inner nested block?

A. No, never
B. Only if declared static
C. Yes, as long as it is declared before the inner block begins
D. Only if declared public

Ans: C

Explanation: Inner blocks can see variables of the enclosing block declared earlier in the same method.

Q8 | Scope Rules | Scope Conflict

Which of the following is illegal in Java regarding scope?

A. Declaring a local variable with the same name as a class-level instance variable
B. Declaring two local variables with the same name in two separate, non-nested methods
C. Declaring a local variable with the same name as another local variable in the same or an enclosing active block
D. Declaring a parameter with the same name as a class-level instance variable

Ans: C

Explanation: Java does not allow re-declaring an identifier that is already in scope within the same or an enclosing block; however, shadowing an instance variable by a local one is permitted.

Q9 | Scope Rules | Shadowing

When a local variable has the same name as an instance variable, this is called:

A. Shadowing
B. Overloading
C. Overriding
D. Casting

Ans: A

Explanation: The local variable temporarily hides (shadows) the instance variable of the same name within its scope.

Q10 | Scope Rules | Method Parameter Scope

The scope of a method parameter is limited to:

A. The entire class
B. The entire package
C. The body of the method in which it is declared
D. Only the first statement of the method

Ans: C

Explanation: Method parameters behave like local variables and exist only within that method's body.

Q11 | Scope Rules | For-loop Scope

In `for(int i=0;i<5;i++){ }`, after the loop ends, can `i` be used outside the loop?

A. Yes, i retains its last value

B. Yes, but only as a static variable

C. No, i is scoped only to the for-loop

D. Yes, if declared final

Ans: C

Explanation: A variable declared in the for-loop's initialization is scoped to the loop itself and is not visible after it ends.

Q12 | Scope Rules | Scope Identification

```
class Demo {
int x = 10;
void show(){
int x = 20;
System.out.println(x);
}
}
```

What will `show()` print?

A. 20

B. 10

C. Compilation error

D. 0

Ans: A

Explanation: The local variable x shadows the instance variable x, so the local value 20 is printed within the method.

Section 2 — Static Data

10 Questions

Q13 | Static Data | Meaning of Static Data

A static data member in a class is:

A. Shared by all objects of the class

B. Created separately for every object

C. Accessible only inside constructors

D. Automatically final

Ans: A

Explanation: Static data members belong to the class itself, so a single copy is shared among all instances.

Q14 | Static Data | Class-level Association

How is a static variable associated with a class?

A. It is created only when the first object is created

B. It is created once per object

C. It is loaded once when the class is loaded, independent of object creation

D. It is never loaded into memory

Ans: C

Explanation: Static members are allocated memory at class-loading time, not at object-creation time.

Q15 | Static Data | Shared Nature

If one object changes the value of a static variable, what happens to other objects?

A. All objects see the updated value since it is shared

B. Other objects retain the old value

C. A new copy is created for each object

D. It causes a runtime exception

Ans: A

Explanation: Since only one copy of a static variable exists, a change through any object is visible to all objects.

Q16 | Static Data | Access and Usage

A static variable can be accessed using:

A. Only the class name
B. Only an object reference
C. Neither, only within static methods
D. Both the class name and an object reference

Ans: D

Explanation: Although recommended access is via the class name, Java also permits access through an object reference.

Q17 | Static Data | Static vs Instance

Which statement correctly distinguishes static and instance data?

A. Static data has one copy per object; instance data has one copy per class
B. Both have one copy per object
C. Both have one copy per class
D. Static data has one copy per class; instance data has one copy per object

Ans: D

Explanation: This is the fundamental distinction: static = per class, instance = per object.

Q18 | Static Data | Declaration

Which keyword is used to declare a static data member?

A. final
B. static
C. const
D. shared

Ans: B

Explanation: The `static` keyword marks a data member as belonging to the class rather than instances.

Q19 | Static Data | Code-based

```
class Counter {
static int count = 0;
Counter(){ count++; }
}
```

If 3 objects of Counter are created, what is the value of count?

A. 1
B. 0
C. Compilation error
D. 3

Ans: D

Explanation: Each constructor call increments the single shared static variable count, resulting in 3 after three objects.

Q20 | Static Data | Default Values

What is the default value of a static int field if not explicitly initialized?

A. null
B. Garbage value
C. Compilation error
D. 0

Ans: D

Explanation: Static fields, like instance fields, receive default values automatically; for int this default is 0.

Q21 | Static Data | Static Final

A variable declared as `static final` typically represents:

A. A variable that changes per object
B. A constant shared by the entire class
C. A variable that cannot be accessed
D. A local variable

Ans: B

Explanation: Combining static with final creates a single, unchangeable, class-wide constant.

Q22 | Static Data | Access Restriction

Can a static variable be accessed from a non-static (instance) method?

A. No, never
B. Only using reflection
C. Only if declared public
D. Yes, without any restriction

Ans: D

Explanation: Instance methods can freely access static members because they exist independent of any particular object.

Section 3 — Static Methods

10 Questions

Q23 | Static Methods | Meaning

A static method belongs to:

A. A specific object instance only
B. The package only
C. The interface only
D. The class, not to any particular object

Ans: D

Explanation: Static methods are associated with the class and can be invoked without creating an object.

Q24 | Static Methods | Invocation

How is a static method typically invoked?

A. ClassName.methodName()
B. object.new methodName()
C. this.methodName() only
D. super.methodName() only

Ans: A

Explanation: Static methods are conventionally called using the class name, though calling via an object reference also compiles.

Q25 | Static Methods | Access to Static Data

Can a static method directly access static data members of its class?

A. No, it must create an object first
B. Yes, directly
C. Only through reflection
D. Only if the data is final

Ans: B

Explanation: Static methods can freely access static data because both belong to the class, not to any instance.

Q26 | Static Methods | Restriction

Which of the following CANNOT be used directly inside a static method?

A. Static variables of the class
B. Non-static (instance) variables of the class
C. Local variables declared within the method
D. Static methods of the class

Ans: B

Explanation: Instance members require an object to exist, but a static method may run without any object, so direct access is disallowed.

Q27 | Static Methods | this keyword

Why can't the `this` keyword be used inside a static method?

A. Because `this` is a reserved keyword only for constructors
B. Because static methods cannot have parameters
C. Because `this` is only valid in interfaces
D. Because `this` refers to the current object, and a static method may be called without any object existing

Ans: D

Explanation: Static methods are class-level and do not operate in the context of an instance, so `this` has no valid meaning there.

Q28 | Static Methods | Static vs Instance Methods

Which of these correctly differentiates static and instance methods?

A. An instance method can be called without creating an object; a static method requires an object
B. Both require creating an object
C. Neither requires an object
D. A static method can be called without creating an object; an instance method requires an object

Ans: D

Explanation: This is the core operational difference between the two kinds of methods.

Q29 | Static Methods | Overloading Static Methods

Can static methods be overloaded?

A. No, static methods cannot be overloaded
B. Yes, based on differing parameter lists
C. Only if they return different types
D. Only in abstract classes

Ans: B

Explanation: Overloading depends on method signature (name and parameters), not on the static modifier, so static methods can be overloaded.

Q30 | Static Methods | Code Output

class MathOps {
static int square(int n){ return n*n; }
}
What does `MathOps.square(5)` return?

A. 10
B. 25
C. 5
D. Compilation error

Ans: B

Explanation: The static method simply computes and returns n*n, i.e., 5*5 = 25.

Q31 | Static Methods | main Method

Why is the `main` method declared static in Java?

A. Because main cannot take arguments otherwise
B. Because static methods run faster
C. Because main must return an int
D. So the JVM can call it without creating an object of the class first

Ans: D

Explanation: The JVM invokes main() before any object of the class exists, which requires main to be static.

Q32 | Static Methods | Static Method Overriding

Can a static method be truly overridden (polymorphically) in Java?

A. Yes, exactly like instance methods
B. Yes, but only with the same return type
C. Yes, if declared final
D. No, static methods are hidden, not overridden, in a subclass

Ans: D

Explanation: Redeclaring a static method in a subclass results in method hiding, resolved at compile time, not runtime polymorphism.

Section 4 — Static Blocks

10 Questions

Q33 | Static Blocks | Meaning

A static block in a class is used to:

A. Initialize instance variables when an object is created
B. Initialize static data members when the class is loaded
C. Define the main method
D. Declare abstract methods

Ans: B

Explanation: Static blocks run once, at class-loading time, mainly to perform static initialization.

Q34 | Static Blocks | Syntax

Which is the correct syntax for a static block?

A. block static { // code }
B. static void { // code }
C. static { // code }
D. final static() { // code }

Ans: C

Explanation: A static block is written using the keyword `static` followed by a block of code in braces.

Q35 | Static Blocks | Execution Timing

When does a static block execute?

A. Every time an object is created
B. Every time a method is called
C. Once, when the class is first loaded into memory
D. Only when main() explicitly calls it

Ans: C

Explanation: Static blocks execute a single time during class loading, before any object is created.

Q36 | Static Blocks | Order of Execution

If a class has a static block and a static variable declared before it, in what order are they executed?

A. Static blocks always run before static variable initializations regardless of position

B. Static variables always run after all static blocks

C. The order is undefined

D. In the order they appear in the source code

Ans: D

Explanation: Static initializers (variables and blocks) execute in the textual order in which they appear in the class.

Q37 | Static Blocks | Multiple Static Blocks

If a class has multiple static blocks, how are they executed?

A. In reverse order

B. In the order they are written, top to bottom

C. Only the last one executes

D. Only the first one executes

Ans: B

Explanation: Java allows multiple static blocks, and they all execute sequentially in their source-code order.

Q38 | Static Blocks | Class Loading

Static blocks are primarily related to which stage of class lifecycle?

A. Object destruction

B. Class loading/initialization

C. Garbage collection

D. Method invocation

Ans: B

Explanation: Static initialization, including static blocks, is performed as part of the class-loading process by the JVM.

Q39 | Static Blocks | Code/Output

```
class Test {
static int x;
static { x = 10; System.out.println("Block1"); }
static { System.out.println("Block2: "+x); }
public static void main(String[] a){ System.out.println("Main"); }
}
```

What is printed?

A. Block1, Block2: 10, Main

B. Main, Block1, Block2: 10

C. Block2: 10, Block1, Main

D. Block1, Block2: 0, Main

Ans: A

Explanation: Static blocks run in order before main() begins, so Block1 sets x and prints, then Block2 prints x=10, then Main runs.

Q40 | Static Blocks | Purpose

A common purpose of a static block is to:

A. Initialize instance variables for each object separately

B. Create objects of the class automatically

C. Perform complex initialization of static variables that cannot be done in a single expression

D. Override methods of the superclass

Ans: C

Explanation: Static blocks allow multi-statement or conditional logic to initialize static state, unlike a simple assignment.

Q41 | Static Blocks | Static Block vs Method

A static block differs from a static method mainly because:

A. A static block executes automatically at class loading, while a static method must be explicitly called
B. A static block can be called explicitly like a method
C. A static method executes automatically at class loading
D. A static block can accept parameters and return values

Ans: A

Explanation: Static blocks have no name and cannot be invoked directly; they run once automatically, unlike static methods.

Q42 | Static Blocks | Restrictions

Can a static block directly access an instance (non-static) variable of the class?

A. Yes, always
B. No, it cannot, since no object exists yet at class-loading time
C. Yes, but only using `this`
D. Yes, but only in the last static block

Ans: B

Explanation: Static blocks execute before any object exists, so instance variables (tied to objects) are not accessible directly.

Section 5 — Class, Method, Data Member and Variable Modifiers

15 Questions

Q43 | Class, Method, Data Member and Variable Modifiers | Class Modifiers

Which modifier can be applied to a top-level class to prevent it from being extended?

A. private
B. protected
C. final
D. static

Ans: C

Explanation: A `final` class cannot be subclassed; this restricts inheritance of that class.

Q44 | Class, Method, Data Member and Variable Modifiers | Class Modifiers

Which access modifier(s) can be applied to a top-level (outer) class?

A. private or protected only
B. public, private, and protected
C. public or default (no modifier) only
D. static only

Ans: C

Explanation: Top-level classes can only be public or have default (package-private) access; private and protected are not allowed for top-level classes.

Q45 | Class, Method, Data Member and Variable Modifiers | Abstract Class Modifier

Which modifier, when applied to a class, means it cannot be instantiated directly?

A. abstract
B. final
C. static
D. public

Ans: A

Explanation: An `abstract` class cannot have objects created directly from it; it must be subclassed.

Q46 | Class, Method, Data Member and Variable Modifiers | Invalid Combination

Which of the following class modifier combinations is invalid?

A. public abstract class
B. public final class
C. abstract final class
D. abstract class (no other modifier)

Ans: C

Explanation: A class cannot be both abstract (must be subclassed) and final (cannot be subclassed) at the same time.

Q47 | Class, Method, Data Member and Variable Modifiers | Method Modifiers

Which modifier prevents a method from being overridden in a subclass?

A. static
B. abstract
C. final
D. public

Ans: C

Explanation: Declaring a method `final` locks its implementation, disallowing subclasses from overriding it.

Q48 | Class, Method, Data Member and Variable Modifiers | Method Modifiers

An abstract method must be declared in which type of class?

A. An abstract class or an interface
B. A final class
C. A static nested class only
D. Any concrete class

Ans: A

Explanation: Abstract methods have no body and can only appear in abstract classes or interfaces, which need not provide implementation.

Q49 | Class, Method, Data Member and Variable Modifiers | Invalid Method Declaration

Which method declaration is invalid?

A. public abstract void show();
B. public static void show(){}
C. abstract final void show();
D. protected final void show(){}

Ans: C

Explanation: abstract and final are contradictory on a method: abstract requires overriding, final forbids it.

Q50 | Class, Method, Data Member and Variable Modifiers | Data Member Modifiers

Which modifier makes a data member accessible only within its own class?

A. public
B. protected
C. private
D. default (no modifier)

Ans: C

Explanation: A `private` member can be accessed only from within the same class in which it is declared.

Q51 | Class, Method, Data Member and Variable Modifiers | Data Member Modifiers

Which modifier allows a data member to be accessed by subclasses in other packages, but not by unrelated classes?

A. private
B. default (no modifier)
C. static
D. protected

Ans: D

Explanation: `protected` grants access to the same package and to subclasses even in different packages.

Q52 | Class, Method, Data Member and Variable Modifiers | Variable Modifier

Which modifier makes a variable's value unchangeable once assigned?

A. static
B. private
C. abstract
D. final

Ans: D

Explanation: A `final` variable can be assigned only once; any further reassignment causes a compile-time error.

Q53 | Class, Method, Data Member and Variable Modifiers | Default Access

When no access modifier is specified for a class member, what is its access level called?

A. Public access
B. Default (package-private) access
C. Private access
D. Protected access

Ans: B

Explanation: Absence of an explicit modifier gives package-private (default) access, visible only within the same package.

Q54 | Class, Method, Data Member and Variable Modifiers | Local Variable Modifiers

Which modifier can be legally applied to a local variable?

A. public
B. protected
C. private
D. final

Ans: D

Explanation: Local variables can only be declared `final` among access-related modifiers; public/private/protected are not permitted for locals.

Q55 | Class, Method, Data Member and Variable Modifiers | Static + Final

A field declared `public static final` is typically used to represent:

A. A variable unique to each object
B. A globally accessible constant
C. A method that cannot be overridden
D. A private helper field

Ans: B

Explanation: This combination gives class-wide, unchangeable, and universally accessible values — the common way constants are declared.

Q56 | Class, Method, Data Member and Variable Modifiers | Effect of static on Method

What effect does the `static` modifier have on a method?

A. It makes the method belong to the class, callable without an object
B. It prevents the method from being called at all
C. It makes the method abstract
D. It restricts the method to the same package only

Ans: A

Explanation: Adding `static` associates the method with the class rather than any instance, allowing calls like ClassName.method().

Q57 | Class, Method, Data Member and Variable Modifiers | Identification

Which modifier is applicable to a method to indicate it has no body and must be implemented by subclasses?

A. final
B. static
C. abstract
D. private

Ans: C

Explanation: The `abstract` modifier on a method signals it must be overridden with an implementation in a concrete subclass.

Section 6 — Abstract Classes

12 Questions

Q58 | Abstract Classes | Concept

An abstract class in Java is a class that:

A. Contains only abstract methods and no implementation at all
B. Can always be instantiated using `new`
C. May contain both abstract and concrete methods and cannot be instantiated directly
D. Cannot have constructors

Ans: C

Explanation: Abstract classes can mix declared-only (abstract) methods with fully implemented (concrete) ones, but objects cannot be created from them directly.

Q59 | Abstract Classes | Purpose

What is the main purpose of an abstract class?

A. To prevent any class from ever extending it
B. To provide a common base with partial implementation that subclasses must complete
C. To allow multiple inheritance of implementation
D. To store only static data

Ans: B

Explanation: Abstract classes let related subclasses share common code while forcing them to implement specific abstract behavior.

Q60 | Abstract Classes | Declaration

Which keyword is used to declare an abstract class?

A. abstract
B. interface
C. static
D. final

Ans: A

Explanation: The `abstract` keyword before `class` marks it as an abstract class.

Q61 | Abstract Classes | Abstract Methods

An abstract method is a method that:

A. Has a complete body and can be called directly
B. Can only exist in a final class
C. Is declared without a body and must be implemented by a subclass
D. Is automatically static

Ans: C

Explanation: Abstract methods only declare a signature; the actual implementation is deferred to a concrete subclass.

Q62 | Abstract Classes | Concrete Methods in Abstract Class

Can an abstract class contain fully implemented (concrete) methods?

A. Yes, alongside abstract methods
B. No, all methods must be abstract
C. Only static methods can be concrete
D. Only private methods can be concrete

Ans: A

Explanation: Abstract classes may freely mix abstract method declarations with regular, fully implemented methods.

Q63 | Abstract Classes | Object Creation Restriction

Can you create an object directly from an abstract class using `new`?

A. Yes, always allowed
B. Yes, but only inside main()
C. Yes, if it has no abstract methods declared
D. No, it is not allowed

Ans: D

Explanation: Even an abstract class with zero abstract methods still cannot be instantiated because of the `abstract` keyword itself.

Q64 | Abstract Classes | Subclass Implementation

If a subclass does not implement all abstract methods of its abstract superclass, the subclass must:

A. Be declared final
B. Also be declared abstract
C. Be declared static
D. Automatically inherit default implementations

Ans: B

Explanation: A concrete class must implement every inherited abstract method; otherwise it, too, must remain abstract.

Q65 | Abstract Classes | Abstract vs Concrete Class

Which of the following distinguishes an abstract class from a normal (concrete) class?

A. A concrete class can have abstract methods but an abstract class cannot
B. An abstract class may have unimplemented (abstract) methods; a concrete class cannot
C. Both are identical in every aspect
D. Only concrete classes can have constructors

Ans: B

Explanation: The presence of possible abstract methods and the inability to instantiate is what sets abstract classes apart.

Q66 | Abstract Classes | Inheritance Relationship

Abstract classes are used in inheritance chiefly to:

A. Prevent subclassing entirely
B. Allow objects to be created without any subclass
C. Enforce that subclasses provide specific behavior while sharing common structure
D. Avoid using constructors

Ans: C

Explanation: They establish a contract-like relationship: shared code plus required, subclass-specific implementations.

Q67 | Abstract Classes | Constructors in Abstract Class

Can an abstract class have a constructor?

A. No, abstract classes cannot have constructors
B. Yes, but it can never be called
C. Yes, which is called when a subclass object is instantiated
D. Only static constructors are allowed

Ans: C

Explanation: Abstract classes can define constructors, invoked via `super()` when a concrete subclass object is created.

Q68 | Abstract Classes | Compilation Question

```
abstract class Shape {
abstract void draw();
}
class Circle extends Shape {
}
```

What happens when this code is compiled?

A. Compilation error, because Circle does not implement draw()
B. Compiles successfully
C. Runtime exception when Circle object is created
D. Circle automatically becomes an interface

Ans: A

Explanation: Since Circle is concrete but fails to override the abstract method draw(), the compiler reports an error.

Q69 | Abstract Classes | Common Misconception

Which statement about abstract classes is a common misconception that is actually FALSE?

A. An abstract class must contain at least one abstract method
B. An abstract class can contain instance variables
C. An abstract class can have static methods
D. An abstract class can be used as a reference type

Ans: A

Explanation: An abstract class is not required to declare any abstract method — it can be abstract purely to prevent instantiation.

Section 7 — Interfaces

12 Questions

Q70 | Interfaces | Concept

An interface in Java is best described as:

A. A class that can be instantiated directly
B. A special kind of static block
C. A private helper structure
D. A contract that specifies a set of methods a class must implement

Ans: D

Explanation: Interfaces define what a class must do (its methods) without dictating how, forming a contract.

Q71 | Interfaces | Purpose

What is the main purpose of using interfaces?

A. To achieve abstraction and support multiple inheritance of type
B. To store instance data unique to each object
C. To prevent classes from implementing any methods
D. To replace constructors

Ans: A

Explanation: Interfaces let unrelated classes agree to a common set of behaviors, and a class may implement several interfaces.

Q72 | Interfaces | Declaration

Which keyword is used to declare an interface?

A. abstract
B. class
C. static
D. interface

Ans: D

Explanation: The `interface` keyword is used in place of `class` to define an interface type.

Q73 | Interfaces | Implementation

A class uses which keyword to implement an interface?

A. extends
B. implements
C. imports
D. uses

Ans: B

Explanation: Classes use `implements` to indicate they are fulfilling the contract defined by an interface.

Q74 | Interfaces | Methods in Interface

By default, methods declared in an interface (without a body) are implicitly:

A. private and final
B. public and abstract
C. protected and static
D. public and final

Ans: B

Explanation: Interface methods without a body are, by default, public and abstract, requiring implementation in the implementing class.

Q75 | Interfaces | Variables in Interface

Fields declared in an interface are implicitly:

A. public, static, and final
B. private and static
C. protected and final
D. default access and non-final

Ans: A

Explanation: Any variable in an interface is automatically treated as a constant: public, static, and final.

Q76 | Interfaces | Multiple Interfaces

Can a class implement more than one interface at the same time?

A. No, only one interface per class is allowed
B. Yes, a class can implement multiple interfaces
C. Only abstract classes can implement multiple interfaces
D. Only if the interfaces are in the same package

Ans: B

Explanation: Java allows a class to implement several interfaces, which is how it supports multiple inheritance of type.

Q77 | Interfaces | Abstract Behavior

Since interface methods (without a body) are abstract by nature, what must an implementing class do?

A. Leave them unimplemented
B. Provide a concrete implementation for those methods
C. Declare them private
D. Convert them into static methods

Ans: B

Explanation: A non-abstract class implementing an interface must supply bodies for all its abstract methods.

Q78 | Interfaces | Interface vs Abstract Class

Which of the following is a key difference between interfaces and abstract classes at this syllabus level?

A. An abstract class can be implemented by multiple classes, but an interface cannot
B. Interfaces can have constructors, but abstract classes cannot
C. A class can implement multiple interfaces, but can extend only one class (abstract or not)
D. Abstract classes cannot have any methods at all

Ans: C

Explanation: Java supports single class inheritance but multiple interface implementation, which is a core structural difference.

Q79 | Interfaces | Object Creation

Can an object be created directly from an interface using `new`?

A. Yes, always
B. Yes, but only with no methods declared
C. Yes, if it has only one method
D. No, interfaces cannot be instantiated

Ans: D

Explanation: Like abstract classes, interfaces cannot be instantiated directly since they represent a contract, not a complete implementation.

Q80 | Interfaces | Code Example

```
interface Shape {
void area();
}
class Square implements Shape {
public void area(){ System.out.println("Area"); }
}
```

Will this code compile?

A. No, because interfaces cannot be implemented
B. Yes, because Square correctly implements the abstract method area()
C. No, because area() must be private
D. No, because Square must also extend a class

Ans: B

Explanation: Square provides a public implementation of the interface's abstract method, satisfying the contract, so it compiles.

Q81 | Interfaces | Common Misconception

Which of the following is TRUE regarding interface methods at this syllabus level?

A. An implementing class may ignore any interface methods it does not need

B. Interface methods can have private access by default

C. Interface methods are automatically static

D. An implementing class must override all abstract methods declared in the interface it implements

Ans: D

Explanation: Because interface methods (without a body) are public and abstract by default, the implementing class is obligated to override every one of them.

Section 8 — Inner Classes

10 Questions

Q82 | Inner Classes | Concept

An inner class in Java is:

A. A class defined within another (outer) class

B. A class defined outside all other classes

C. A special kind of interface

D. A class that cannot have methods

Ans: A

Explanation: Inner classes are nested inside an outer class, tying their existence conceptually to that outer class.

Q83 | Inner Classes | Purpose

A common purpose of using inner classes is to:

A. Make classes accessible from any package automatically

B. Avoid ever creating objects of the outer class

C. Logically group classes that are used in only one place, increasing encapsulation

D. Prevent the outer class from having any methods

Ans: C

Explanation: Inner classes tie helper functionality closely to the outer class, hiding it from unrelated code.

Q84 | Inner Classes | Declaration

Where is an inner class physically declared?

A. In a separate file with no relation to any class

B. Inside a method's return statement

C. Inside the package declaration

D. Inside the body of another class

Ans: D

Explanation: An inner (non-static nested) class is written within the braces of its enclosing outer class.

Q85 | Inner Classes | Outer-Inner Relationship

A non-static inner class object is associated with:

A. A specific instance of its outer class

B. No outer class instance at all

C. Every class in the package

D. Only static members of the outer class

Ans: A

Explanation: A non-static inner class instance implicitly holds a reference to the outer class instance that created it.

Q86 | Inner Classes | Creating Inner Class Objects

Which of the following correctly creates an object of a non-static inner class `Inner` from outside the outer class `Outer`?

A. Outer.Inner innerObj = new Inner();

B. Outer.Inner innerObj = new Outer.Inner();

C. Inner innerObj = new Outer().Inner();

D. Outer outerObj = new Outer(); Outer.Inner innerObj = outerObj.new Inner();

Ans: D

Explanation: Because a non-static inner class needs an outer instance, it must be created using outerObjectReference.new Inner().

Q87 | Inner Classes | Access to Outer Members

Can a non-static inner class access the private members of its outer class?

A. No, never

B. Only if they are declared protected

C. Only through getter methods

D. Yes, it can access them directly

Ans: D

Explanation: Inner classes have full access to all members, including private ones, of their enclosing outer class.

Q88 | Inner Classes | Scope of Inner Class

The scope of a non-static inner class is limited to:

A. The entire package regardless of access modifiers

B. Every class in the project

C. The body/scope of its enclosing outer class

D. Only the main() method

Ans: C

Explanation: Its visibility, unless made public, is naturally tied to the enclosing class's scope.

Q89 | Inner Classes | Common Mistake

What is a common mistake when creating an inner-class object?

A. Declaring the inner class as public

B. Giving the inner class a constructor

C. Trying to instantiate a non-static inner class without first creating an outer class object

D. Declaring a method inside the inner class

Ans: C

Explanation: Since a non-static inner class instance needs an enclosing instance, skipping that step causes a compile-time error.

Q90 | Inner Classes | Code Example

```
class Outer {
class Inner {
void show(){ System.out.println("Inner class"); }
}
}
```

Which statement correctly creates and uses an Inner object in another class' main() method?

A. Outer.Inner i = new Outer.Inner(); i.show();

B. Inner i = new Inner(); i.show();

C. Outer.Inner i = new Inner(new Outer()); i.show();

D. Outer o = new Outer(); Outer.Inner i = o.new Inner(); i.show();

Ans: D

Explanation: The correct syntax requires an outer object first, then using it to instantiate the inner class with `.new`.

Q91 | Inner Classes | Inner vs Outer Class

Which best differentiates an inner class from its outer class?

A. The outer class always depends on the inner class instance
B. Both must always be declared static
C. The inner class is nested within, and often depends on, the outer class instance; the outer class stands independently
D. Neither can access the other's members

Ans: C

Explanation: Inner classes are dependent constructs tied to the outer class, whereas the outer class can exist and be used independently.

Section 9 — Wrapper Classes

10 Questions

Q92 | Wrapper Classes | Concept

A wrapper class in Java is used to:

A. Replace all primitive types entirely
B. Store only static final constants
C. Define new primitive types
D. Represent a primitive data type as an object

Ans: D

Explanation: Wrapper classes wrap a primitive value inside an object, enabling it to be used where objects are required.

Q93 | Wrapper Classes | Purpose

Wrapper classes are especially useful when:

A. Primitive performance needs to be maximized in loops
B. You want to avoid using classes altogether
C. Primitive values need to be treated as objects, e.g., in collections
D. You need to declare an interface

Ans: C

Explanation: Many Java APIs (like collections, prior to widespread generics-with-primitives support) require objects, so wrapper classes bridge that gap.

Q94 | Wrapper Classes | Primitive-Wrapper Pairs

Which wrapper class corresponds to the primitive type `int`?

A. Int
B. Integer
C. Integar
D. Num

Ans: B

Explanation: The wrapper class for `int` is named `Integer`, following Java's standard wrapper naming.

Q95 | Wrapper Classes | Primitive-Wrapper Pairs

Which wrapper class corresponds to the primitive type `char`?

A. Character
B. Char
C. Charactor
D. Chr

Ans: A

Explanation: The primitive `char` maps to the wrapper class `Character`.

Q96 | Wrapper Classes | Primitive-Wrapper Pairs

Which wrapper class corresponds to the primitive type `boolean`?

A. Bool
B. Boolen
C. Boolean
D. Bit

Ans: C

Explanation: `boolean` is wrapped by the class `Boolean`.

Q97 | Wrapper Classes | Primitive-Wrapper Pairs

Which wrapper class corresponds to the primitive type `double`?

A. Decimal
B. Double
C. Float
D. Real

Ans: B

Explanation: The primitive `double` maps to the wrapper class `Double` (note: `float` maps separately to `Float`).

Q98 | Wrapper Classes | Identification

Which of the following is NOT a Java wrapper class?

A. Number
B. Integer
C. Long
D. Byte

Ans: A

Explanation: `Number` is an abstract superclass of the numeric wrapper classes, not itself a primitive's direct wrapper class.

Q99 | Wrapper Classes | Wrapper vs Primitive

Which of the following is TRUE about wrapper classes compared to primitives?

A. Primitives can be stored directly in object-only structures but wrapper classes cannot
B. Wrapper objects can be stored in data structures that require objects, whereas primitives cannot
C. Wrapper classes take up less memory than primitives
D. There is no difference between primitives and wrapper classes

Ans: B

Explanation: Because wrapper classes are objects, they satisfy APIs requiring object references, unlike raw primitives.

Q100 | Wrapper Classes | Basic Conversion

Converting a primitive value into its corresponding wrapper object is generally referred to as:

A. Casting
B. Serialization
C. Overriding
D. Boxing

Ans: D

Explanation: Wrapping a primitive into an object form is called boxing (with auto-boxing being the automatic version of this).

Q101 | Wrapper Classes | Code-based

Integer num = 25;
What does this statement represent?

A. A compilation error since int cannot become Integer
B. Explicit casting of Integer to int
C. Auto-boxing of the primitive int value into an Integer object
D. Declaration of a static variable

Ans: C

Explanation: Java automatically converts (auto-boxes) the primitive 25 into an Integer object during assignment.

Section 10 — Packages

8 Questions

Q102 | Packages | Concept

A package in Java is essentially:

A. A namespace/grouping mechanism that organizes related classes and interfaces
B. A single class containing all program code
C. A synonym for a static method
D. A type of wrapper class

Ans: A

Explanation: Packages group related types together and provide a hierarchical namespace to avoid naming conflicts.

Q103 | Packages | Purpose

One major purpose of using packages is to:

A. Prevent any class from being reused
B. Avoid class name conflicts and organize code logically
C. Force all classes to be public
D. Eliminate the need for access modifiers

Ans: B

Explanation: Packages act as containers that keep related classes together and distinguish identically named classes in different packages.

Q104 | Packages | Package Declaration

Which statement correctly declares that a class belongs to package `mypack`?

A. import mypack;
B. using mypack;
C. namespace mypack;
D. package mypack;

Ans: D

Explanation: The `package` keyword, followed by the package name and a semicolon, must be the first statement in the source file.

Q105 | Packages | Position of Package Statement

Where must the package declaration statement appear in a Java source file?

A. Anywhere in the file
B. As the very first statement, before any import or class declaration
C. Only at the end of the file
D. Immediately before the main method

Ans: B

Explanation: Java requires the package declaration, if present, to be the first non-comment line of the file.

Q106 | Packages | Naming

Java package names are conventionally written in:

A. All uppercase letters
B. All lowercase letters
C. CamelCase starting with uppercase
D. Numbers only

Ans: B

Explanation: By convention, package names use lowercase to avoid conflicts with class names, which typically start uppercase.

Q107 | Packages | Benefits

Which of these is a benefit of using packages?

A. Elimination of all compilation errors
B. Automatic conversion of primitives to wrapper classes
C. Guaranteed thread safety
D. Better access control through package-level visibility

Ans: D

Explanation: Packages allow package-private access, letting classes in the same package cooperate closely while hiding details from outside.

Q108 | Packages | Packages and Classes

The relationship between a package and the classes within it is best described as:

A. A package is a subclass of every class it contains
B. A package acts as a container holding one or more related classes and interfaces
C. Each class must belong to multiple packages simultaneously
D. Classes cannot exist without belonging to a package with the same name as the class

Ans: B

Explanation: Packages are organizational containers; a class belongs to exactly one package (default package if unspecified).

Q109 | Packages | Default Package

If a Java file has no package statement, the class belongs to:

A. The java.lang package automatically
B. No package at all, causing a compile error
C. The default (unnamed) package
D. Every package simultaneously

Ans: C

Explanation: Classes without an explicit package declaration are placed in Java's default, unnamed package.

Section 11 — Package Access

8 Questions

Q110 | Package Access | Meaning

'Package access' (default access, no modifier) means a member is accessible:

A. To all classes in any package
B. Only to classes within the same package
C. Only within the same class
D. Only to subclasses regardless of package

Ans: B

Explanation: Default access restricts visibility to classes that reside in the same package as the member's class.

Q111 | Package Access | Within Same Package

A class with default (package) access can be used by:

A. Any class in any package
B. Any other class in the same package
C. Only subclasses in other packages
D. No other class at all

Ans: B

Explanation: Default access allows unrestricted use within the same package but not from outside it.

Q112 | Package Access | Access From Outside Package

Can a class from a different package access a member declared with default (no modifier) access?

A. Yes, always
B. Yes, but only if it's static
C. Yes, but only through inheritance
D. No, it cannot

Ans: D

Explanation: Default access is strictly limited to the same package; classes in other packages cannot access such members, even subclasses.

Q113 | Package Access | Access and Visibility

How does package access relate to encapsulation?

A. It exposes all members to every class regardless of package
B. It restricts visibility to trusted classes within the same package, supporting encapsulation
C. It has no relation to encapsulation
D. It only affects static methods

Ans: B

Explanation: By limiting visibility to the package, default access helps hide implementation details from unrelated external code.

Q114 | Package Access | Code-based Access

package pkgA;
class Sample {
int value = 5; // default access
}
Can a class in `package pkgB` directly access `value`?

A. No, because default access is limited to the same package
B. Yes, directly
C. Yes, if pkgB imports pkgA.*
D. Yes, if the accessing class is public

Ans: A

Explanation: Since `value` has default access, only classes physically inside pkgA can access it, regardless of imports.

Q115 | Package Access | Valid Scenario

Which scenario represents VALID access to a default-access member?

A. A subclass in a different package directly using the member
B. A totally unrelated class in a different package using the member
C. A class in the same package directly using the member
D. Any class using it via wildcard import from another package

Ans: C

Explanation: Only same-package classes can validly access default-access members; even subclasses elsewhere cannot.

Q116 | Package Access | Invalid Scenario

Which is an INVALID access scenario for a default-access method?

A. A class in the same package calling the method
B. The class itself calling its own method
C. A class in a different package, with no inheritance relation, calling the method
D. A subclass in the same package calling the method

Ans: C

Explanation: Default access forbids any class outside the declaring package from calling the method, related or not.

Q117 | Package Access | Comparison

How does package (default) access differ from protected access?

A. Protected access also allows subclasses in different packages, while default access does not
B. Default access allows subclasses in different packages, while protected does not
C. Both are identical in every respect
D. Protected access is more restrictive than default access

Ans: A

Explanation: Protected extends default access by additionally permitting subclass access across package boundaries.

Section 12 — Importing Packages and Classes

8 Questions

Q118 | Importing Packages and Classes | Purpose

The `import` statement in Java is used to:

A. Physically copy source code from another package
B. Declare which package the current class belongs to
C. Allow a class to use classes from another package without fully qualifying their names
D. Prevent access to classes in other packages

Ans: C

Explanation: Import statements let you refer to classes by their simple name instead of writing the full package path every time.

Q119 | Importing Packages and Classes | Importing a Class

Which statement correctly imports the class `Scanner` from `java.util`?

A. import java.util.Scanner;
B. import java.util.Scanner();
C. package java.util.Scanner;
D. using java.util.Scanner;

Ans: A

Explanation: A single-class import specifies the fully qualified class name ending with the class itself, followed by a semicolon.

Q120 | Importing Packages and Classes | Importing Multiple Classes

How can multiple specific classes from the same package be imported in Java?

A. By separating class names with commas in a single import statement
B. It is not possible to import more than one class
C. By writing a separate import statement for each class
D. By using the `multiimport` keyword

Ans: C

Explanation: Java requires one import statement per class name; there is no comma-separated multi-class import syntax.

Q121 | Importing Packages and Classes | Wildcard Import

What does `import java.util.*;` do?

A. Imports all publicly accessible classes and interfaces directly within the java.util package
B. Imports java.util and all its sub-packages recursively
C. Imports only one randomly chosen class
D. Causes a compilation error

Ans: A

Explanation: The wildcard `*` imports all top-level types in that specific package, but not its sub-packages.

Q122 | Importing Packages and Classes | Basic Syntax

Where must `import` statements appear in a Java source file?

A. After all class declarations
B. After the package statement (if any) and before any class declaration
C. Inside the main method only
D. Anywhere inside a method body

Ans: B

Explanation: Import statements come after an optional package declaration and before the class/interface definitions.

Q123 | Importing Packages and Classes | Package vs Import

What is the key difference between a package declaration and an import statement?

A. package specifies which package the current file belongs to, whereas import brings in classes from other packages
B. import specifies which package the current file belongs to, whereas package brings in outside classes
C. Both statements do exactly the same thing
D. package is optional but import is mandatory in every file

Ans: A

Explanation: `package` defines the file's own package membership, while `import` allows use of external classes by simple name.

Q124 | Importing Packages and Classes | Compilation-related

If a class uses `java.util.ArrayList` without importing it and without using the fully qualified name, what happens?

A. It compiles fine automatically
B. It runs but throws a runtime exception
C. Java assumes it is a user-defined class
D. A compilation error occurs, since the compiler cannot resolve the simple name

Ans: D

Explanation: Without an import or a fully qualified reference, the compiler cannot locate the ArrayList class and reports an error.

Q125 | Importing Packages and Classes | Fully Qualified Name

Instead of importing, a class can also be used by writing its:

A. Package name alone
B. Class name in lowercase
C. Fully qualified name, e.g., java.util.ArrayList
D. Interface name

Ans: C

Explanation: A fully qualified name includes the entire package path, letting you use the class without any import statement.

Section 13 — User-Defined Packages

8 Questions

Q126 | User-Defined Packages | Concept

A user-defined package is:

A. A package pre-built into the Java standard library
B. A package that cannot contain more than one class
C. A synonym for the default package
D. A package created by the programmer to organize their own classes

Ans: D

Explanation: Unlike built-in packages like java.util, user-defined packages are custom namespaces created by developers.

Q127 | User-Defined Packages | Creating a Package

What is the first step to create a user-defined package?

A. Create an interface with the package's name
B. Add a `package packageName;` statement at the top of the source file
C. Add a static block naming the package
D. Import java.lang explicitly

Ans: B

Explanation: Declaring the package name as the very first statement places the class into that user-defined package.

Q128 | User-Defined Packages | Directory Structure

For a class declared as `package shapes;`, where should its compiled .class file typically reside for it to be found correctly?

A. In any folder regardless of name
B. In a folder named after the class only
C. In a folder named `shapes`, matching the package name
D. Directly in the root with no folder

Ans: C

Explanation: Java's classloader expects the directory structure to mirror the package name for proper resolution.

Q129 | User-Defined Packages | Organizing Classes

User-defined packages help programmers by:

A. Automatically making all classes public
B. Grouping logically related custom classes together, separate from other packages
C. Preventing classes from having constructors
D. Removing the need for the main method

Ans: B

Explanation: Just like built-in packages, user-defined ones let developers organize their own code into logical, reusable units.

Q130 | User-Defined Packages | Accessing Classes

How can a class in package `pkg1` access a public class in a user-defined package `pkg2`?

A. By importing pkg2's class or using its fully qualified name
B. Access is impossible across user-defined packages
C. By declaring the class private
D. By renaming the class to match pkg1

Ans: A

Explanation: Public classes in user-defined packages are accessed the same way as classes in built-in packages: via import or fully qualified name.

Q131 | User-Defined Packages | Importing User-Defined Classes

Which statement correctly imports a class `Employee` from a user-defined package `company`?

A. import company.*.Employee;

B. package company.Employee;

C. import company.Employee;

D. using company.Employee;

Ans: C

Explanation: The import syntax for a user-defined package's class is identical to that for a built-in package's class.

Q132 | User-Defined Packages | Basic Example

package school;
public class Student { }
Which of the following correctly uses this class from another package?

A. import school.Student; ... Student s = new Student();

B. Student s = new Student(); (with no import, in a class from a different package)

C. package school.Student; Student s = new Student();

D. school Student s = new Student();

Ans: A

Explanation: Without importing or fully qualifying, a class from a different package cannot directly reference Student.

Q133 | User-Defined Packages | Common Mistake

A common mistake when creating user-defined packages is:

A. Naming the package in lowercase

B. Declaring the class as public

C. Forgetting to place the package statement as the very first line of the file

D. Compiling the file at all

Ans: C

Explanation: If the package statement isn't the first statement, or is missing when expected, the class will not belong to the intended package, causing errors.

Section 14 — Comparison Questions

11 Questions

Q134 | Comparison Questions | Local vs Member Scope

Which correctly compares local scope and member (instance) scope?

A. Local scope is limited to the block/method of declaration; member scope spans the whole class

B. Member scope is limited to a single method; local scope spans the whole class

C. Both scopes are identical in extent

D. Local scope variables are shared by all objects while member scope is not

Ans: A

Explanation: Local variables live only within their declaring block, while instance variables are visible throughout the class (subject to access modifiers).

Q135 | Comparison Questions | Static vs Instance Data

Which correctly compares static data and instance data?

A. Instance data has a single shared copy; static data has a separate copy per object

B. Static data has a single copy shared by the class; instance data has a separate copy per object

C. Both always share a single copy

D. Neither is stored in memory until referenced

Ans: B

Explanation: This is the defining conceptual contrast between class-level and object-level data storage.

Q136 | Comparison Questions | Static vs Instance Method

Which correctly compares static and instance methods?

A. An instance method can be invoked without an object; a static method needs one

B. Both require an object to be invoked

C. A static method can be invoked without an object; an instance method needs an object to be invoked

D. Neither can access static data

Ans: C

Explanation: Static methods operate at class level and don't require object creation, unlike instance methods.

Q137 | Comparison Questions | Static Block vs Method

Which correctly compares a static block and a static method?

A. A static method runs automatically once at class loading; a static block must be called explicitly

B. Both run automatically at class loading

C. A static block runs automatically once at class loading; a static method runs only when explicitly called

D. Both must be called explicitly by name

Ans: C

Explanation: Static blocks have no name and cannot be invoked; they execute automatically, unlike named static methods.

Q138 | Comparison Questions | Abstract vs Concrete Class

Which correctly compares an abstract class and a concrete class?

A. An abstract class may contain unimplemented methods and cannot be instantiated; a concrete class must implement all methods and can be instantiated

B. A concrete class may contain unimplemented methods; an abstract class cannot

C. Both can always be instantiated directly

D. Neither can have constructors

Ans: A

Explanation: This captures the essential contrast: partial vs. complete implementation, and instantiability.

Q139 | Comparison Questions | Abstract Class vs Interface

Which correctly compares an abstract class and an interface at this syllabus level?

A. A class can extend multiple abstract classes but implement only one interface

B. Both allow multiple inheritance of implementation equally

C. A class can extend only one abstract class but implement multiple interfaces

D. Neither can be used as a reference type

Ans: C

Explanation: Java restricts class inheritance to single abstract/concrete classes, while permitting multiple interface implementation.

Q140 | Comparison Questions | Inner Class vs Outer Class

Which correctly compares an inner class and its outer class?

A. The outer class always depends on an inner class instance

B. Both are always independent of each other

C. Neither can access the other's members

D. A non-static inner class instance typically depends on an outer class instance; the outer class can exist independently

Ans: D

Explanation: The dependency is generally one-directional: inner classes rely on the outer instance, not vice versa.

Q141 | Comparison Questions | Primitive vs Wrapper Class

Which correctly compares a primitive type and its wrapper class?

A. A wrapper class stores a raw value directly; a primitive stores it inside an object

B. Both are identical in memory representation

C. Primitives can be used wherever objects are required, but wrappers cannot

D. A primitive stores a raw value directly; a wrapper class stores the value inside an object

Ans: D

Explanation: Wrapper classes exist precisely to represent primitive values as full-fledged objects.

Q142 | Comparison Questions | Package vs Class

Which correctly compares a package and a class?

A. A class is a container that holds multiple packages

B. A package is a container that can hold multiple classes; a class is a single unit of code within a package

C. A package can only contain one class

D. Both are exactly the same construct

Ans: B

Explanation: Packages organize and group classes; a class is one member of that grouping.

Q143 | Comparison Questions | Package Declaration vs Import

Which correctly compares a package declaration and an import statement?

A. import declares which package the current file belongs to; package brings in outside classes

B. Both statements always appear multiple times per file

C. Neither can appear in the same file together

D. package declares which package the current file belongs to; import brings in classes from other packages for use

Ans: D

Explanation: These are complementary but distinct statements serving different roles in a Java source file.

Q144 | Comparison Questions | Package Access vs Outside Access

Which correctly compares access within a package versus access from outside a package (for default-access members)?

A. Within the same package, default-access members are freely accessible; from outside the package, they are not accessible at all

B. From outside the package, default-access members are freely accessible; within the package, they are not

C. Both cases allow full access

D. Both cases disallow all access

Ans: A

Explanation: Default access is deliberately package-scoped, granting access inside but denying it entirely to outside classes.

Section 15 — Code, Output and Compilation Questions

12 Questions

Q145 | Code, Output and Compilation Questions | Output

```
class Test {
static int a = 5;
public static void main(String[] args){
int a = 10;
System.out.println(a);
}
}
```

What is printed?

A. 5

B. 15

C. Compilation error

D. 10

Ans: D

Explanation: The local variable a inside main() shadows the static field a, so the local value 10 is printed.

Q146 | Code, Output and Compilation Questions | Valid Declaration

Which of these class declarations is VALID?

A. public abstract final class Animal { abstract void sound(); }

B. public abstract class Animal { abstract void sound(); }

C. private class Animal { }

D. abstract final class Animal { }

Ans: B

Explanation: abstract and final together on a class are contradictory, and top-level classes cannot be private, so only the first option is valid.

Q147 | Code, Output and Compilation Questions | Modifier Applicability

Which modifier is applicable to a local variable but NOT to a top-level class?

A. static can apply to both equally

B. public can apply to both equally

C. abstract can apply to both equally

D. final can apply to both, but private cannot apply to a top-level class

Ans: D

Explanation: private is disallowed on top-level classes, while final is valid on both local variables and classes; the distinguishing restricted modifier here is private.

Q148 | Code, Output and Compilation Questions | Variable Accessibility

```
class Box {
private int size;
void setSize(int size){ this.size = size; }
}
```

Which variable does `this.size` refer to inside setSize?

A. The instance variable `size` of the class

B. The parameter `size`

C. A new local variable

D. Undefined behavior

Ans: A

Explanation: `this.size` explicitly refers to the instance variable, disambiguating it from the parameter of the same name.

Q149 | Code, Output and Compilation Questions | Static Block Execution

```
class Loader {
static { System.out.println("Loading"); }
Loader(){ System.out.println("Object created"); }
}
public class Test {
public static void main(String[] a){
Loader l1 = new Loader();
Loader l2 = new Loader();
}
}
```

What is printed?

A. Loading, Object created, Object created

B. Loading, Loading, Object created, Object created

C. Object created, Object created, Loading

D. Loading, Object created

Ans: A

Explanation: The static block runs once when the class is loaded, and the constructor runs once per object created (twice here).

Q150 | Code, Output and Compilation Questions | Method Call Validity

```
class Util {
static void greet(){ System.out.println("Hi"); }
}
```

Can `greet()` be called as `Util.greet();` from another class?

A. No, static methods can never be called from another class

B. Yes, since it is a static method accessible via the class name

C. Only if Util is declared abstract

D. Only if greet() is declared public and Util has an object created first

Ans: B

Explanation: Assuming default/public access permits it, a static method can be invoked directly with ClassName.method().

Q151 | Code, Output and Compilation Questions | Object Creation Check

```
abstract class Vehicle {
abstract void move();
}
```

Which statement is INVALID?

A. Vehicle v = new Car(); // where Car extends Vehicle and implements move()

B. abstract class Vehicle{ } (no other change)

C. Vehicle v = new Vehicle();

D. class Car extends Vehicle { void move(){ } }

Ans: C

Explanation: You cannot instantiate an abstract class directly, even though you can reference it via a concrete subclass object.

Q152 | Code, Output and Compilation Questions | Interface Implementation Check

```
interface Movable {
void move();
}
class Robot implements Movable {
public void move(){ System.out.println("Moving"); }
}
```

Which class correctly implements the interface?

A. Robot, but only if move() is declared private

B. Robot, because it overrides move() with public access

C. No class can correctly implement Movable without extending it too

D. Robot only if it also declares move() as abstract

Ans: B

Explanation: Interface methods are implicitly public and abstract, so implementing classes must override them with public access.

Q153 | Code, Output and Compilation Questions | Inner Class Declaration Validity

```
class Outer {
class Inner { }
}
public class Test {
public static void main(String[] a){
Outer.Inner obj = new Outer().new Inner();
}
}
```

Is this declaration valid?

A. No, Inner classes can never be instantiated from outside Outer

B. No, `new Outer().new Inner()` is invalid syntax

C. Yes, this is the correct way to instantiate a non-static inner class from outside

D. Only valid if Inner is declared static

Ans: C

Explanation: This is the standard syntax: create the outer instance, then use it with `.new` to create the inner instance.

Q154 | Code, Output and Compilation Questions | Wrapper Identification

double d = 10.5;
Double dObj = d;
What does the second line represent?

A. A compilation error

B. Explicit unboxing

C. Static initialization

D. Auto-boxing of the primitive double into a Double wrapper object

Ans: D

Explanation: Java automatically converts the primitive double value into a Double object through auto-boxing.

Q155 | Code, Output and Compilation Questions | Package Declaration Validity

Is the following valid?
import java.util.*;
package myapp;
public class Test { }

A. Yes, order does not matter

B. No, because the package statement must come before any import statement

C. Yes, but only if Test is abstract

D. Yes, because import statements can appear anywhere

Ans: B

Explanation: The package declaration, if present, must always be the very first statement in the file, before any imports.

Q156 | Code, Output and Compilation Questions | Import Validity

Is `import java.util.ArrayList, java.util.HashMap;` valid Java syntax?

A. Yes, comma-separated imports are valid

B. Yes, but only inside a static block

C. Yes, if both classes are in the same package

D. No, each class must be imported using a separate import statement

Ans: D

Explanation: Java does not support importing multiple classes with a single comma-separated import statement.

Section 16 — Difficult & Tricky Questions

10 Questions

Q157 | Difficult & Tricky Questions | Tricky Scope

```
class A {
int x = 1;
void method(){
int x = x + 1; // Line X
System.out.println(x);
}
}
```

What happens at Line X?

A. Prints 2

B. Prints 1

C. Prints 0

D. Compilation error, because the local variable x is used in its own initializer before being definitely assigned

Ans: D

Explanation: The local x shadows the instance x from the point of declaration, so `x + 1` refers to the not-yet-initialized local x, causing an error.

Q158 | Difficult & Tricky Questions | Tricky Static

```
class Demo {
static int count;
static { count = 10; }
static { count += 5; }
public static void main(String[] args){ System.out.println(count); }
}
```

What is printed?

A. 10

B. 5

C. 15

D. Compilation error

Ans: C

Explanation: Both static blocks execute in order at class loading: first sets count to 10, second adds 5, giving 15 before main() runs.

Q159 | Difficult & Tricky Questions | Tricky Modifier

Which of these is a valid combination for a method modifier?

A. abstract static void show();

B. private abstract void show();

C. final abstract void show();

D. public static final void show(){}

Ans: D

Explanation: abstract requires the method to be overridden, which conflicts with static (class-bound, hidden not overridden), private (not inheritable), and final (cannot be overridden).

Q160 | Difficult & Tricky Questions | Tricky Abstract

```
abstract class Base {
void display(){ System.out.println("Base display"); }
}
class Derived extends Base { }
public class Test {
public static void main(String[] a){
Base b = new Derived();
b.display();
}
}
```

What is the output?

A. Compilation error, because Base is abstract

B. Base display

C. Nothing is printed

D. Runtime exception

Ans: B

Explanation: Even though Base is abstract and cannot be instantiated directly, referencing a Derived object via a Base reference and calling an inherited concrete method works fine.

Q161 | Difficult & Tricky Questions | Tricky Interface

```
interface A { void show(); }
interface B { void show(); }
class C implements A, B {
public void show(){ System.out.println("C's show"); }
}
```

Is this valid?

A. No, a class cannot implement two interfaces with the same method name

B. Yes, because both interfaces declare the same abstract method signature, one implementation satisfies both

C. No, this always causes ambiguity errors

D. Only valid if A and B are in the same package

Ans: B

Explanation: Since both interface methods share an identical signature, a single override in C satisfies both contracts without conflict.

Q162 | Difficult & Tricky Questions | Tricky Inner Class

```
class Outer {
private int x = 100;
class Inner {
void show(){ System.out.println(x); }
}
}
public class Test {
public static void main(String[] a){
Outer.Inner in = new Outer().new Inner();
in.show();
}
}
```

What is printed?

A. 100

B. Compilation error, x is private

C. 0

D. Runtime exception

Ans: A

Explanation: Inner classes have full access to the private members of their enclosing outer class, so x=100 is accessible and printed.

Q163 | Difficult & Tricky Questions | Tricky Wrapper

```
Integer a = 100;
Integer b = 100;
System.out.println(a == b);
```

What is printed, and why (conceptually, at this syllabus level)?

A. true, because small boxed integer values are commonly cached and may reference the same object

B. false, because == always compares object identity and never matches for wrapper objects

C. Compilation error

D. true, because == always compares values for wrapper classes

Ans: A

Explanation: Java caches boxed Integer objects for a small value range, so `==` can return true for values in this cached range; this is a well-known tricky behavior for wrapper classes.

Q164 | Difficult & Tricky Questions | Tricky Package Access

```
package pkgA;
class Helper {
void assist(){ System.out.println("Assisting"); }
}
package pkgB;
import pkgA.Helper;
public class Test {
public static void main(String[] a){
Helper h = new Helper();
h.assist();
}
}
```

What is the likely result?

A. Compiles and runs, printing 'Assisting'

B. Runtime exception only

C. Compilation error, because Helper class itself has default access and cannot be used outside pkgA

D. Compiles but prints nothing

Ans: C

Explanation: Even though Helper is imported, its own class-level default access restricts usage to classes within pkgA; importing does not override that restriction.

Q165 | Difficult & Tricky Questions | Tricky Static Method Hiding

```
class Parent {
static void greet(){ System.out.println("Parent"); }
}
class Child extends Parent {
static void greet(){ System.out.println("Child"); }
}
public class Test {
public static void main(String[] a){
Parent p = new Child();
p.greet();
}
}
```

What is printed?

A. Parent

B. Child

C. Compilation error

D. Both Parent and Child

Ans: A

Explanation: Static methods are resolved based on the reference type at compile time (method hiding), not the actual object type, so Parent's greet() runs.

Q166 | Difficult & Tricky Questions | Tricky Final Variable

```
class Test {
final int x;
Test(){ x = 10; }
public static void main(String[] a){
Test t = new Test();
System.out.println(t.x);
}
}
```

Is this valid, and what is printed?

A. Compilation error, because final variables must be initialized at declaration

B. Invalid, because final fields cannot be assigned in constructors

C. Prints 0, since final fields always default to 0

D. Valid; prints 10, because a blank final variable can be assigned exactly once in the constructor

Ans: D

Explanation: A 'blank final' instance variable is allowed as long as it is definitely assigned exactly once, such as in every constructor.

Section 17 — Mixed OAVS TGT Practice Set

10 Questions

Q167 | Mixed OAVS TGT Practice Set | Mixed

Which of the following members belongs to the class rather than to individual objects?

A. A static data member

B. An instance variable

C. A local variable inside a constructor

D. A method parameter

Ans: A

Explanation: Static data members are associated with the class itself and shared across all its objects.

Q168 | Mixed OAVS TGT Practice Set | Mixed

Which modifier combination on a class is definitely invalid?

A. public final
B. public abstract
C. abstract (alone)
D. abstract final

Ans: D

Explanation: abstract and final are mutually exclusive on a class because one requires subclassing and the other forbids it.

Q169 | Mixed OAVS TGT Practice Set | Mixed

A class implementing an interface but not providing bodies for all abstract methods must itself be declared:

A. abstract
B. final
C. static
D. private

Ans: A

Explanation: If not all abstract methods are implemented, the implementing class remains incomplete and must be marked abstract.

Q170 | Mixed OAVS TGT Practice Set | Mixed

Which of the following correctly creates a non-static inner class instance?

A. outerRef.new InnerClass()
B. new InnerClass()
C. new OuterClass.InnerClass()
D. InnerClass.new()

Ans: A

Explanation: A non-static inner class requires an existing outer instance, accessed via the `.new` syntax on that instance.

Q171 | Mixed OAVS TGT Practice Set | Mixed

Which wrapper class is used for the primitive `long`?

A. Large
B. Long
C. LongInt
D. BigLong

Ans: B

Explanation: The wrapper class corresponding to primitive `long` is simply named `Long`.

Q172 | Mixed OAVS TGT Practice Set | Mixed

Which statement about static blocks is correct?

A. They execute once for every object created
B. They execute once, automatically, when the class is loaded
C. They must be explicitly called using the class name
D. They can accept method arguments

Ans: B

Explanation: Static blocks are automatic, unnamed initializers tied strictly to class loading, not object creation.

Q173 | Mixed OAVS TGT Practice Set | Mixed

Which of these is TRUE about default (package) access?

A. It permits access only to classes within the same package
B. It permits access to all classes everywhere
C. It permits access only within the declaring class
D. It is the most permissive access level in Java

Ans: A

Explanation: Default access sits between private and protected in restrictiveness, scoped exactly to the package.

Q174 | Mixed OAVS TGT Practice Set | Mixed

Which import statement is syntactically correct?

A. import java.io.File
B. import java.io.File;
C. include java.io.File;
D. import(java.io.File);

Ans: B

Explanation: A valid import statement uses the `import` keyword, the fully qualified class name, and ends with a semicolon.

Q175 | Mixed OAVS TGT Practice Set | Mixed

Which statement about interfaces is TRUE at this syllabus level?

A. All fields in an interface are implicitly public, static, and final
B. Interface fields can be modified after being set
C. Interfaces can be instantiated with `new`
D. A class can implement only one interface

Ans: A

Explanation: Interface fields are always treated as constants: public, static, and final by default.

Q176 | Mixed OAVS TGT Practice Set | Mixed

In `package shapes;`, if the class `Circle` is declared `public`, how should another package's class access it?

A. Directly, without any import, from any package
B. It cannot be accessed since it belongs to a user-defined package
C. Only if it is also declared static
D. By importing `shapes.Circle` or using the fully qualified name `shapes.Circle`

Ans: D

Explanation: Public classes in user-defined packages follow the same access rules as any built-in package class: import or full qualification.